SENG 4220

Lab 8 and 9: Intercepting Communication

Total Challenges: 19 (Challenge 00 – Challenge 18)

Overview

These two labs introduces you to core network interception and manipulation techniques through a series of independent Capture The Flag (CTF) style challenges. You will practice essential skills used in network security, including host discovery, traffic capture and analysis, packet crafting and spoofing at multiple layers (Ethernet, IP, TCP, UDP, and ARP), firewall configuration, and basic man-in-the-middle operations.

The challenges are independent. You may complete them in any order. There is no difficulty progression. Each challenge focuses on a specific concept or technique.

All challenges run inside isolated Docker containers. In every challenge, your goal is to capture the flag by successfully performing the required network operation.

Learning Objectives

By completing this assignment, you will be able to:

- Use common Linux networking tools effectively, including `ping`, `nmap`, `tcpdump`, `nc`, `curl`, `ip`, `ifconfig`, and `iptables`
- Capture and analyze live network traffic using `tcpdump`
- Craft and send custom packets at different layers of the network stack using `Scapy` and raw sockets
- Perform host discovery and network scanning
- Configure firewall rules to allow or block specific traffic
- Execute basic spoofing attacks (Ethernet frames, IP packets, TCP/UDP packets, and ARP)
- Perform simple man-in-the-middle attacks using ARP poisoning
- Understand and apply concepts such as traffic interception, session/cookie hijacking, and denial-of-service techniques

---

How to Run the Challenges

To get started with the challenges, you will use the provided Docker environment. This ensures all tools and dependencies are correctly set up and provides a secure, isolated workspace for your tasks.

Prerequisites Ensure you have Docker installed and running on your machine (e.g., Docker Desktop for Windows/Mac, or Docker Engine for Linux).

Step 1: Setup Your Environment Download the provided assignment ZIP file and extract it. Ensure that the main files (including `launcher.py`, pyarmor_runtime_000000 directory, and `intercepting-communication.tar`) are placed together in the exact same folder on your machine.

Step 2: Start Challenge 00 Open your terminal, navigate to your extracted folder, and run the following command:

```
python3 launcher.py 00
```

Step 3: Enter Your University ID When prompted, enter your University ID and press **Enter**. *(Important: Remember to submit this exact ID with your solutions, as your instructor will use it to verify your unique flags during grading).*

Step 4: Let Docker Load (First Time Only) If this is your first time running a challenge, the script will automatically load the required challenge images from the `.tar` archive. Once you see `[✓] Successfully loaded all images!` and the shell prompt drops to something like `hacker@...:~$`, you are fully set up and ready to begin!

Step 5: Test Your Setup To ensure everything is working correctly, test Challenge 00 by typing the following command:

Bash

history

You should see a flag appear. If this works, your setup is complete and fully functional.

Step 6: Running the Actual Challenges To start any other challenge (for example, Challenge 01), simply run the launcher again with the new challenge number:

Bash

python3 launcher.py 01

Inside the container, you will again see the hacker prompt (`hacker@df86723de7ae:~$`). To activate the challenge, run the following command:

Bash

/challenge/run

You will be switched to a privileged environment (usually `root@ip-10-0-0-x:~#`) where the challenge is active. Read the challenge description provided in your assignment document and work on solving it to capture your unique flag!

To get started with the challenges, you will use the provided Docker environment to ensure all tools and dependencies are correctly set up.

Follow these steps:

1. Download all the provided assignment files (including `Dockerfile` and `docker.sh`) and place them together into a single folder on your machine.

2. Open your terminal and navigate to that folder.

3. Start the Docker setup by executing the following command:

   ```
   ./docker.sh
   ```

(Note: If you receive a "Permission denied" error when trying to run the script, make it executable first by running `chmod +x docker.sh` and then try again). 4. The script will automatically build the image and start the container. You should have a terminal similar to this: `sh root@5d4a5e648acf:/workspace#` 5. After running the docker, you should have the following files in your working directory:

```
$ ls
Dockerfile
docker.sh
generate-challenges.py
intercepting-communication.tar
pyarmor_runtime_000000
```

5. Run challenge 00 by

   ```
   python3 generate-challenges.py 00
   ```

   When prompted, enter your *University ID* and press Enter (and remember to submit this id with your solutions so your instructor will use the same exact id for marking).

6. The script will load the required challenge images. Once you see `Successfully loaded all images!` and the shell prompt drops to `hacker@...:~$`, you are fully set up and ready to begin!

7. Test your setup with Challenge 00

   Type the following command:

   ```
   history
   ```

   You should see a flag appear. If this works, your setup is complete and working correctly.

8. **Run the actual challenges**

   To start any challenge (for example Challenge 01), run:

```
python3 generate-challenges.py 01
```

Inside the container, you will again see a prompt like:

```
hacker@df86723de7ae:~$
```

Then run the following command to begin the challenge:

```
/challenge/run
```

You will be switched to a privileged environment (usually `root@ip-10-0-0-x:~#`) where the challenge is active.

9. Read the challenge description inside this assignment and work on solving it to capture the flag.

Important Notes

- **First-time loading**: The first time you run any challenge, Docker will load the images. This may take a few minutes. After the first run, all challenges will start much faster.
- **Always start with Challenge 00** to verify that your environment is set up correctly.

Inspecting Challenge Code (Optional but Recommended)

Inside each challenge container, you can view the source code of the challenge by running:

```
cat /challenge/run
```

This command will show you exactly how the challenge is implemented.

Challenge Environment & Flag Submission

- Each challenge is a **Capture The Flag (CTF)** activity.
- When you successfully complete a challenge, you will receive a flag in this format:

  ```
  seng-4220{challenge-17-flag-for-students}
  ```

- **Do not** modify the provided codes to generate challenges. It is protected and used for grading.
- Use your full university ID as input. The flag will be generated based on that, and the instructor will generate the same flag using your key and their private key.

Important Warnings

- **Always make sure you start your challenge with `challenge/run` and you are in this terminal `root@ip-10-0-0-x:~#` to complete the CTF challenges.**
- You cannot install new packages inside the challenge containers. All required tools have already been pre-installed.
- Work only with the tools and libraries that are already available in the environment.
- Do not try to update or upgrade the system (`apt update`, `apt install`, etc.). These commands will not work or may break the environment.
- Always exit the container cleanly using `exit` when you are done with a challenge.

Available Tools & Resources

It is expected that CS/Software students are comfortable working in a terminal. However, we have provided the following list of common commands and tools for those with limited background. These tools are widely used, and extensive tutorials are available online for each.

It is expected that CS/Software students are comfortable working in a terminal. However, we have provided the following list of common commands and tools for those with limited background. If you are not familiar with these tools or need a refresher, we highly recommend reviewing this custom guide we built for students: https://keshvadi.github.io/themissingseng .

1. Basic Linux Terminal

You should be comfortable with the following commands:

Command	Purpose	Example
`pwd`	Show current directory	`pwd`
`ls`	List files	`ls -la`
`cd`	Change directory	`cd /challenge`
`cat`	View file content	`cat file.txt`
`vim / nano`	Edit files	`vim script.py`
`python3`	Run Python scripts	`python3 script.py`

chmod	Change file permissions	`chmod +x script.sh`

2. Common Networking Tools

Tool	What it is	Why we use it	Basic Example
ping	Tests connectivity	Check if a host is reachable	`ping 10.0.0.2`
nc	Netcat – read/write network data	Connect or listen on ports	`nc 10.0.0.2 31337`
nmap	Network scanner	Discover hosts and open ports	`nmap 10.0.0.0/24`
tcpdump	Packet capture tool	Capture and analyze traffic	`tcpdump -i eth0`
curl	Transfer data using URLs	Interact with web services	`curl http://10.0.0.2`
ip	Show/manipulate network interfaces	Configure IP addresses	`ip addr show`
ifconfig	Older network interface tool	View interface information	`ifconfig`
iptables	Firewall / NAT configuration	Block or redirect traffic	`iptables -L`
python3	Python interpreter	Write custom network scripts	`python3 -c "print('Hello')"`

3. Scapy Quick Start

To work with low-level networking tasks such as crafting and sending custom packets, you will need to write small Python programs. Python 3 is pre-installed in the environment, and the powerful networking library `Scapy` is available for this purpose. `Scapy` makes it easy to create, manipulate, and transmit packets across a wide range of protocols. One basic example is provided below; for more advanced usage and additional examples, refer to the official tutorial linked in the resources section.

Basic Examples:

```
from scapy.all import *

# Create and send an ICMP packet
pkt = IP(dst="10.0.0.2") / ICMP()
send(pkt)

# Create a TCP packet
tcp_pkt = IP(dst="10.0.0.2") / TCP(dport=80, flags="S")
send(tcp_pkt)

# ARP Request
arp_pkt = ARP(pdst="10.0.0.2")
send(arp_pkt)
```

```
# Simple UDP packet
udp_pkt = IP(dst="10.0.0.2") / UDP(dport=53) /
DNS(qd=DNSQR(qname="google.com"))
send(udp_pkt)
```

Recommended Resources:

- **Official Tutorial (Starting Scapy):** If you are new to Scapy or have limited experience with packet manipulation in Python, we highly recommend starting here. It provides a quick, practical introduction to get you up to speed.
- **Full Documentation (https://scapy.readthedocs.io):** This is intended as a reference for all students. You do not need to read it end-to-end, but you should keep it open to look up specific packet fields, functions, and syntax as you work through the challenges.

Challenges

There are **19 challenges** in total (Challenge 00 to Challenge 18).
Challenge 00 is an introductory challenge designed to help you understand the environment and find the first flag.

Full descriptions, objectives, and hints for each challenge are provided at the end of this document.

Submission Instructions

1. Complete each challenge and obtain the flag.
2. Submit the flag on the course submission platform.
3. You may attempt the challenges in any order and any number of times that you want.

Academic Integrity

- You may discuss general concepts with classmates.
- You must solve and submit flags **individually**.
- Do not share flags or complete solutions.

Important Ethical and Legal Warning

The Linux systems used in these challenges have no security features disabled. All hosts and networks you interact with are running standard, up-to-date Linux installations with their default protections enabled.

This means the techniques you will practice in this assignment are realistic and can be used against real systems if those systems lack proper defenses.

You must never use any of the tools or techniques from this assignment on networks, systems, or services that you do not own or have explicit written permission to test.

Doing so may be illegal and can have serious consequences. **[Production: Please ensure this paragraph jumps out at the students.]**

Even on networks you own or manage, running these types of tools without proper authorization is often against organizational policy and may be restricted by law. You will learn more about the legal and ethical aspects of network security in Unit 4.

Want to Practice More?

If you enjoyed these challenges and would like to continue practicing, there are many excellent Capture The Flag platforms available online. These sites offer a wide range of more advanced and sophisticated challenges that can help you further develop your network security and penetration testing skills.

Some popular platforms include: - TryHackMe - Hack The Box - PicoCTF - CTFtime (for finding upcoming CTF events)

Good luck!

Challenge Descriptions & Hints

Challenge 01 – Host Discovery with Ping

In this challenge you will be attempting to connect to a remote host. You must first run `/challenge/run` in order to access the network: `/challenge/run` will drop you into a shell on a host with network access. From your host at 10.0.0.1, connect to some unknown remote host on the 10.0.0.0/24 subnet, on port 31337.

Fortunately, there are only 256 possible hosts on this subnet, so you can just try them all!

One simple tool that you can use to help you with this is `ping`. If you "ping" a host, and it is up, you will get a response; otherwise, ping will timeout and warn you that it cannot reach the host.

For example, try pinging yourself:

```
ping 10.0.0.1
```

This will continuously keep pinging until you press `Ctrl-C` to stop it.

You can also try pinging a host that you know is down:

```
timeout 10 ping 10.0.0.2
```

or sending a few packets using `-c`:

```
ping -c 1 10.0.0.2
```

This will run `ping` for (up to) 10 seconds (or send only one packet in the second case), but you should see messages from `ping` indicating that the host is unreachable before the timeout.

As with most commands, you can also run `man ping` to see the manual page for `ping`.

Consider this an opportunity to practice your shell scripting skills! You can of course ping each of the 256 hosts manually, but maybe a `for` loop might be even easier!

```
for i in $(seq 10); do
  echo $i
done
```

Important: Don't forget to run /challenge/run to access the network or else you will not find the remote host.

Challenge 02 – Network Scanning with Nmap

The goal of this challenge is to be able to find the live hosts in a network.

From your host at 10.0.0.1, connect to some unknown remote host on the 10.0.0.0/16 subnet, on port 31337.

This network is quite big. There are 65,536 possible hosts on this subnet, so finding the remote host manually would really be a pain. Even a basic `for` loop processing 10 hosts per second would take over an hour to complete! We can of course get fancier with our shell scripting (parallelizing, etc.), but for now, let's consider a standard tool designed to help with this kind of task: `nmap`.

`nmap` is a powerful network scanning tool that can be used to discover hosts and services on a computer network. You can, for example, scan for which hosts are up (and popular services running on those hosts) on 10.0.0.0/30 with the following command:

```
nmap 10.0.0.0/30
```

Within 15 seconds or so, you should see that your host at 10.0.0.1 is up, as expected. When conducting a network scan, it is important to be aware of the potential impact on the network. Under default settings, `nmap` tries to be at least somewhat polite and not totally overwhelm a network with tons of packets. Nevertheless, it is still possible to cause network congestion or even trigger security alerts by running a network scan, and so it is important to be aware of the potential impact. As such, you shouldn't run a network scan on a network that you don't own or have permission to scan! In this network, it's okay to be a little more aggressive, a little more "rude" with our scanning.

You'll want to review the man page for `nmap` (`man nmap`) to see how you can speed up the scanning process: you're specifically interested in how many packets are being sent per second. Disabling some of the default scans, such as DNS resolution, can also speed up the scanning process. When in doubt, use `-v` to see a bit more information about what `nmap` is currently doing.

When you figure out which host is alive, use `nc` to connect to that host on port 31337. If you successfully connect, you will receive a flag.

HINT:

You will likely want to use the Wireshark tool for this. However, in the command line you don't have Wireshark. The Linux tool for this is `tcpdump`.

This is installed on the system, and you can launch it from the terminal of the `10.0.0.1` client!

Make sure to launch it from there: if launched from elsewhere (such as a different terminal on the workspace), `tcpdump` will not be running on the correct host!

If you simply run `tcpdump`, you will see all traffic from all IP addresses on all ports. However, viewing the packet payload is not enabled by default.

It is a good idea to familiarize yourself with `tcpdump` in this challenge, since it is a tool that is widely used in networking and network security, and you will use it in future challenges as well.

Challenge 04 – Cookie Hijacking via Traffic Sniffing

In this challenge, you will learn to sniff traffic and steal a cookie from the host machine, then use it to connect to another host.

In the web security unit, you will learn more about cookies. When you authenticate to a server, you do not need to authenticate every time you send a subsequent request. The client (host machine) simply attaches the cookie to each request so the server knows it is the authenticated user. If someone obtains this cookie, they can send requests to the server to impersonate the original host machine. As long as the cookie is valid, the server will grant access. This attack is known as cookie hijacking (or session hijacking when the cookie contains a session identifier).

In this challenge, a web server is running on IP address `10.0.0.2`. When you start the challenge, a client begins sending requests to the web server and attaches the cookie to each request. Your job is to steal the admin's cookie and use it to GET the flag!

If you send a request to `http://10.0.0.2/flag` with the correct cookie, the web server will return the flag.

Hints:

- Use `tcpdump` again to capture the traffic (as in the previous challenge).
- The web server is likely running on port 80, 8080, or 8000.
- Running `tcpdump` without filters will show a lot of traffic because the admin host sends many requests, and each HTTP request contains multiple lines.
- The HTTP request header containing the cookie starts with `Cookie:` and has this format:

  ```
  Cookie: name=value
  ```

  Example:

  ```
  Cookie: session=eyJ1c2VyIjoiYWRtaW4ifQ.ajYaww
  ```

- You need to extract this session cookie value from the captured packets.

- Once you have the cookie value, use `curl` to send a request to the server while attaching the cookie in the header.
- `curl` is a very handy tool for web development, networking, cloud, and security tasks. If you are not familiar with it, take some time to learn the basics (for example, run `man curl` or consult a quick reference).
- **Important:** Perform all steps from the `10.0.0.1` terminal to ensure you are working on the correct host.

Challenge 05 – Traffic Interception via IP Aliasing

In this challenge, you need to steal traffic that is supposed to go to another host and receive it on your own host.

When you join a network, the other systems on the network direct packets to you if the destination address matches your address.

However, your system can also be configured to claim additional IP addresses.

In this task, `10.0.0.2` is sending a flag to `10.0.0.3` on port `31337`. You need to configure your network interface to also receive traffic destined for `10.0.0.3` and capture the flag.

Hints:

- To view your own IP address(es), the network interfaces you are connected to, the subnet, and other useful information, use the `ifconfig` tool (legacy) or the newer `ip` command.
- You can simply run `ifconfig` (or `ip addr`) to see your IP addresses and related details.
- It is a good idea to familiarize yourself with these tools.
- After you configure your network interface to receive traffic destined for `10.0.0.3`, listen on port `31337` to capture the flag.

Challenge 06 – Blocking Traffic with iptables

This challenge and the two following challenges are about the firewall.

Your host at `10.0.0.1` is receiving traffic on port `31337`. Your job is to block that traffic.

To configure your firewall, you need to use the `iptables` tool.

If you correctly block traffic on that specific port, a flag will appear on your terminal.

Challenge 07 – IP-based Firewall Rules with iptables

In this challenge, you set up your firewall to block traffic from a specific IP address.

Your host at `10.0.0.1` is receiving traffic on port `31337`. Block that traffic, but only from the remote host at `10.0.0.3`. You must allow traffic from the remote host at `10.0.0.2`.

If you correctly configure your firewall, you will immediately receive a flag on your terminal.

Challenge 08 – Allowing Outbound Traffic with iptables

In this challenge, outbound traffic to port `31337` has been blocked, so you will need to allow it first.

Your job is to connect from your host at `10.0.0.1` to the remote host at `10.0.0.2` on port `31337`.

After you correctly allow only that traffic, connect to the server on that port to receive the flag.

Challenge 09 – Denial of Service (DoS) Attack

In this challenge, you will conduct a DoS attack on a server.

The client at `10.0.0.3` is communicating with the server at `10.0.0.2` on port `31337`. Deny this service.

Hints:

You need to overwhelm the server with a large amount of traffic. You have already used a tool in previous challenges that connects to a specific port on a specific IP address. You also learned how to write shell scripts to automate tasks. Use these tools together to perform the attack.

If you successfully take down the server, you will receive a flag on your terminal.

Challenge 10 – Ethernet Frame Spoofing

In this challenge, you are going to spoof a link-layer frame.

As you know, systems on a subnet use the link layer to send frames to each other. Instead of IP addresses, they use MAC addresses.

Your task is to manually send an Ethernet frame. The frame should have `EtherType=0xFFFF` and should be sent to the remote host at `10.0.0.2`. Using this technique, you can perform all sorts of link-layer attacks, such as ARP spoofing.

Hints:

- You need to know your own MAC address and the destination host's MAC address to craft a link-layer frame.
- `arping` is a useful tool that works similarly to `ping` but also reveals the destination MAC address.
- The most important part is learning how to craft packets. The Python library for this is **Scapy**. With Scapy you can create a wide variety of networking packets at different layers (you can choose source/destination IP, port, payload, EtherType, and more).
- Scapy is a widely used tool in networking and security. You should familiarize yourself with it, as you will use it in future activities when spoofing packets at different layers.
- If you successfully craft and send an Ethernet frame with `EtherType=0xFFFF` to the target host, you will receive a flag.

- To write code in the terminal, you can use text editors such as `nano`, `vim`, or `emacs`. `vim` is provided for you. If you have not used `vim` before, take a moment to learn the basics.
- Python 3 is also provided, and the required packages are already installed.

Challenge 11 – IP Packet Spoofing

In this challenge, you will spoof an IP-layer packet (similar to the previous Ethernet frame challenge). Such crafted IP packets can be used in various IP-level attacks or DDoS attacks.

Your task is to manually send an Internet Protocol packet. The packet should have `IP protocol=0xFF` and should be sent to the remote host at `10.0.0.2`.

Hints:

Use the same tools and workflow as in the previous challenge: - Use `vim` (or another editor) to write your Python script. - Python 3 and the required packages are already installed. - You can use **Scapy** (as introduced previously) or raw sockets to craft and send the packet. Scapy makes it easier to set fields like protocol, source/destination IP, and payload. - If you successfully craft and send the IP packet with protocol `0xFF`, you will receive a flag.

Challenge 12 – TCP Packet Spoofing

In this challenge, we move one layer higher in the network stack: you will spoof a TCP packet. Such crafted TCP packets can be used in various TCP-based attacks.

Your task is to manually send a Transmission Control Protocol packet. The packet should have: - `TCP sport=31337, dport=31337, seq=31337, ack=31337, flags=APRSF` - It should be sent to the remote host at `10.0.0.2`.

Use the same tools and workflow as in the previous challenges (Ethernet and IP):

If you successfully craft and send the TCP packet with the specified fields, you will receive a flag.

Challenge 13 – UDP Packet Crafting

TCP, as you know, is excellent for reliable, ordered communication over a single connection. However, all of that reliability comes at a performance cost due to handshakes, acknowledgments, retransmissions, and so on.

To address this, the designers of the Internet created **UDP** (User Datagram Protocol). UDP is much simpler than TCP. A UDP header only contains the source port, destination port, length, and a checksum. There is no concept of a "connection," no handshakes, and no guaranteed delivery or ordering.

This simplicity comes with trade-offs: each UDP packet is independent, and any connection-like behavior must be implemented by the application itself. When programming with UDP sockets, there is no `listen()` or `accept()`. Instead, you typically use `recvfrom()` (which returns both the data and the sender's address) and `sendto()`

(which sends data to a specific address). A single server can therefore handle many clients concurrently, but this also requires more care to avoid mixing up messages.

In this challenge, you will make your first UDP connection. UDP spoofing is the most widely type of DDoS attack.

From your host at `10.0.0.1`, send a UDP packet to the remote host at `10.0.0.2` on port `31337` containing the message `Hello, World!\n`. Your code should both send the packet **and** receive the response (which will contain the flag).

You can use either **Scapy** or `netcat`, but we strongly recommend **Scapy** because it will be more useful in future challenges.

Hints:

- Refer to the previous challenges where you used Scapy to craft packets.
- Scapy makes sending and receiving UDP packets very straightforward.
- You can also use the standard `socket` library with `SOCK_DGRAM` if you prefer raw sockets.
- Your program should send the message and then wait for (and print) the response containing the flag.

Challenge 14 – Custom Source Port in UDP Packets

Though we didn't explore this for TCP, both TCP and UDP allow you to choose the **source port** in addition to the destination port.

In this challenge, you will practice setting a custom source port. You can do this with `s.bind()` on the socket (the same way a server binds to a listening port).

From your host at `10.0.0.1`, send a UDP packet **from source port `31338`** to the remote host at `10.0.0.2` on port `31337` containing the message `Hello, World!\n`. Your code must both send the packet **and** receive the response (which will contain the flag).

Important Note:
You **must** call `bind()` to set the source port **before** sending any data. Otherwise, Linux will automatically choose a random ephemeral source port (the default behavior).

Hints:

- Refer to the previous UDP challenge for the basic pattern of sending and receiving with UDP.
- You can use either the standard `socket` library or **Scapy** (recommended for consistency with earlier challenges).
- Remember to bind the socket to your chosen source port before calling `sendto()` or `send()`.

Challenge 15 – UDP Redirection Attack

There is a fairly wide gap between the rich features TCP provides and UDP's bare-bones nature. Sometimes developers want *some* of TCP's features and end up re-implementing just what they need on top of UDP. This can lead to interesting (and sometimes

dangerous) situations, such as the ability to trigger outbound traffic to other servers — a technique that has been abused in UDP-based amplification attacks.

Rather than sending the flag directly to you, this challenge lets you **redirect** it to another server. Your goal is to catch the flag on the other side.

How it works:

When `10.0.0.2` receives a UDP message that starts with `FLAG`, it parses the message in the format `FLAG:host:port` and sends the actual flag to that host and port.

Your task:

Spoof a UDP packet that appears to come from `10.0.0.3:31337` and is sent to `10.0.0.2:31338`, containing the payload:

```
FLAG:10.0.0.1:31337
```

If successful, `10.0.0.2` will send the flag to your host (`10.0.0.1`) on port `31337`. You must receive it.

Hints:

- You can either run a UDP listener on port `31337` (using Python or `netcat`) to catch the flag, **or** simply use `tcpdump` to sniff the traffic when it arrives.
- To run a command in the background, add `&` at the end (e.g., `python3 listener.py &`). You can later bring it to the foreground or kill it with `fg` or `kill`.
- By default, `tcpdump` does **not** show packet payloads. Use the `-A` flag to print the payload in ASCII.
- You already know how to spoof packets with **Scapy** from previous challenges. Use IP spoofing + UDP to make the packet appear to come from `10.0.0.3`.

Challenge 16 – UDP Source Port Attack

Of course, the previous spoofing attack worked because you knew the exact source port the client was using. This allowed you to forge a believable response/request.

This technique was at the heart of a famous vulnerability in the Domain Name System (DNS), the Kaminsky attack, which allowed attackers to forge DNS responses and redirect users to malicious sites. The main fix was to randomize the source port used by DNS resolvers.

In this challenge, the source port is no longer fixed at `31338`. Can you still force the flag to be sent to you?

How the system works (recap): When `10.0.0.2` receives a UDP message starting with `FLAG`, it interprets it as `FLAG:host:port` and sends the actual flag to that host and port.

Your task is to spoof a UDP packet that appears to come from `10.0.0.3` (on its unknown/random source port) to `10.0.0.2`, containing:

```
FLAG:10.0.0.1:31337
```

Hints:

- The source port is chosen once per socket (either explicitly with `bind()` or automatically on the first `sendto()`).
- You can use **Scapy** to rapidly send many spoofed packets.
- You should run a listener (Python or `netcat`) or `tcpdump` on port `31337` on your host to catch the flag when it arrives.
- Running commands in the background with `&` (as shown in the previous challenge) will be useful here.

Challenge 17 – ARP Spoofing Attack

In this challenge, you will conduct an **ARP spoofing** (also known as ARP poisoning) attack.

Your task is to manually send an Address Resolution Protocol (ARP) packet. The packet should inform the remote host that the IP address `10.0.0.42` can be found at the Ethernet (MAC) address `42:42:42:42:42:42`.

The packet should be sent to the remote host at `10.0.0.2`.

Hints:

- ARP operates at the link layer and is used to map IP addresses to MAC addresses.
- You can use **Scapy** (as in previous challenges) to easily craft and send ARP packets.
- To make the spoof believable to the target, you will typically send the packet as a broadcast (`ff:ff:ff:ff:ff:ff`).
- The `op="is-at"` field in Scapy corresponds to an ARP Reply (as opposed to a Request).

If you successfully send the spoofed ARP packet, you should receive a flag.

Challenge 18 – Man-in-the-Middle Attack via ARP Poisoning

In this challenge, you will conduct a **Man-in-the-Middle (MITM)** attack using **ARP poisoning** (also called ARP spoofing).

The remote host at `10.0.0.2` is communicating with the remote host at `10.0.0.3` on port `31337`. Your goal is to intercept this traffic by positioning yourself in the middle.

Your tasks are:

1. Convince `10.0.0.2` that the IP address `10.0.0.3` belongs to your MAC address.
2. Once the poisoning is successful, listen on port `31337` to receive the flag.

Hints:

- You already learned how to add an IP address to your interface and how to send spoofed ARP packets in previous challenges.
- Run the ARP poisoning script in background.
- ARP poisoning is usually done in a loop because ARP entries expire over time.